

---

# Mastering Software Architecture Patterns and System Design

Architecture is the foundation on which successful systems are built.

— Rahul Rajat Singh —

---

# Course Agenda

- Section 1: Foundations of Software Architecture
  - Understand what architecture truly means, its role in the SDLC, and the quality attributes that define great systems.
- Section 2: Layered & Modular Architecture Patterns
  - Deep-dive into monoliths, layered, hexagonal, clean, and onion architectures — and when to use them.
- Section 3: Distributed Systems & Service Architectures
  - Learn how to scale systems across teams and services: microservices, SOA, micro frontends, and more.
- Section 4: Communication & Integration Patterns
  - Master the protocols and patterns that connect components: API gateways, messaging, resilience techniques.
- Section 5: Security & Governance in Architecture
  - Architect secure systems with OAuth2, Zero Trust, secrets management, and real-world security scenarios.
- Section 6: Event-Driven & Reactive Systems
  - Model responsiveness and decoupling with EDA, CQRS, sagas, and reactive principles.
- Section 7: Cloud-Native & Scalable Architecture Styles
  - Modern infrastructure essentials: serverless, containers, scaling patterns, observability, and edge computing.
- Section 8: UI Architecture Patterns for Apps
  - Architect frontend systems with MVC, MVVM, MVU, offline-first strategies, and platform-specific patterns.
- Section 9: Architectural Decision-Making & Trade-Offs
  - Learn to reason through trade-offs, avoid anti-patterns, and align architecture with business and domain.
- Section 10: Conclusion & Next Steps
  - Wrap up with practical next steps, key resources, and strategies to grow your career as a software architect.

---

# What is Software Architecture?

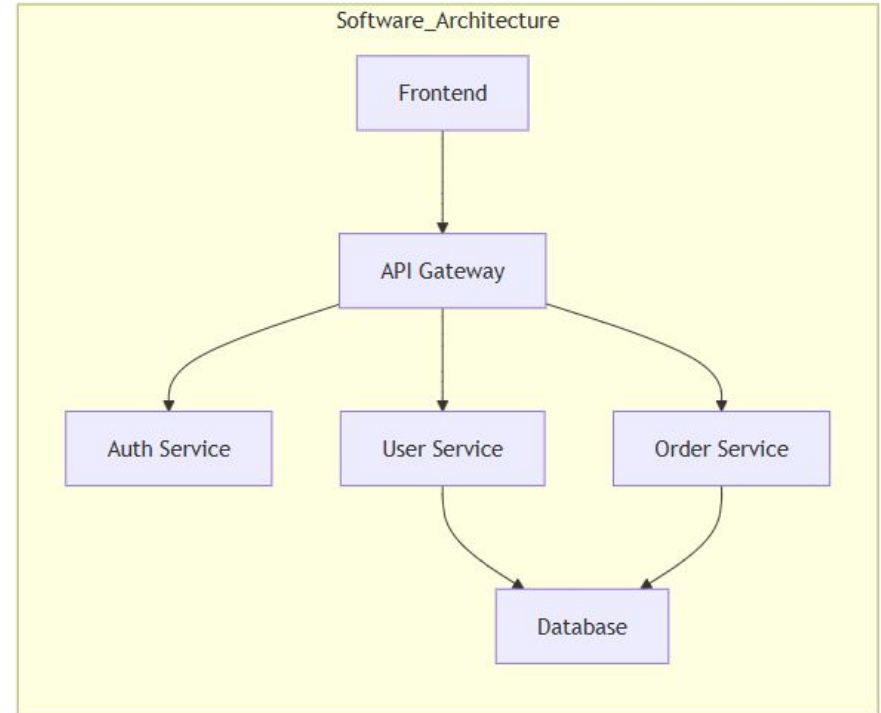
## Foundations of Software Architecture

Mastering Software Architecture Patterns and System Design

---

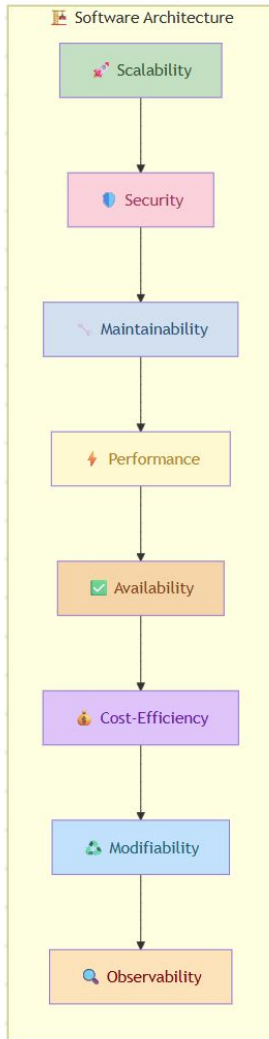
# Definition of Software Architecture

- “Software architecture refers to the fundamental structures of a software system and the discipline of creating such structures.”
- It involves high-level design decisions about components, their responsibilities, and their interactions.



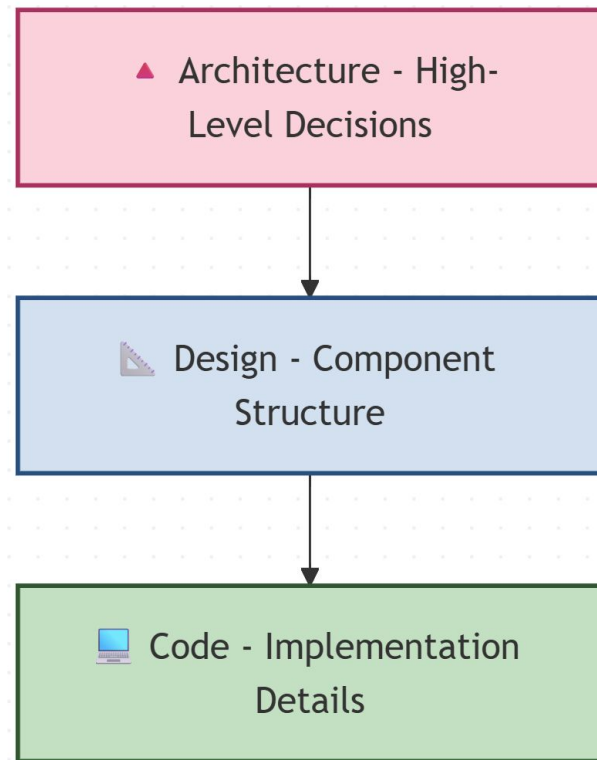
# Goals of Software Architecture

- Provide a structured solution that meets technical and business expectations.
- Support scalability, reliability, performance, security, and maintainability.
- Allow for decision-making clarity at high abstraction levels.



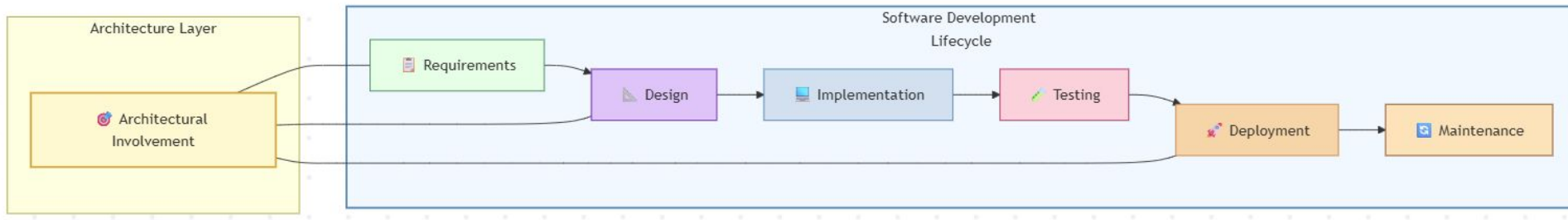
# Architecture vs. Engineering vs. Design

- Architecture = High-level structure & constraints
- Design = Mid-level patterns & interactions
- Code = Detailed implementation



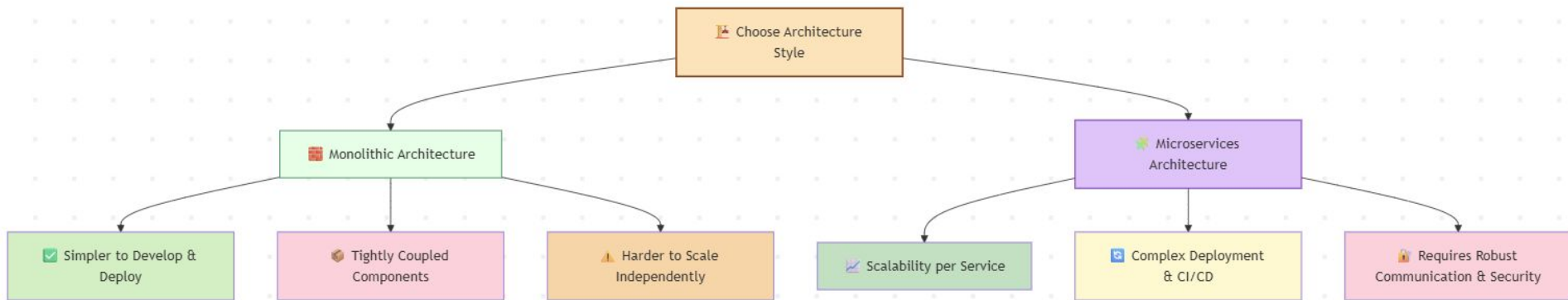
# Architecture in the SDLC

- Plays a role throughout the Software Development Life Cycle
- Impacts:
  - Requirements (e.g., quality attributes)
  - Design decisions
  - Technology selection
  - Risk management
  - DevOps and deployment



# Architecture is about Structure + Decisions

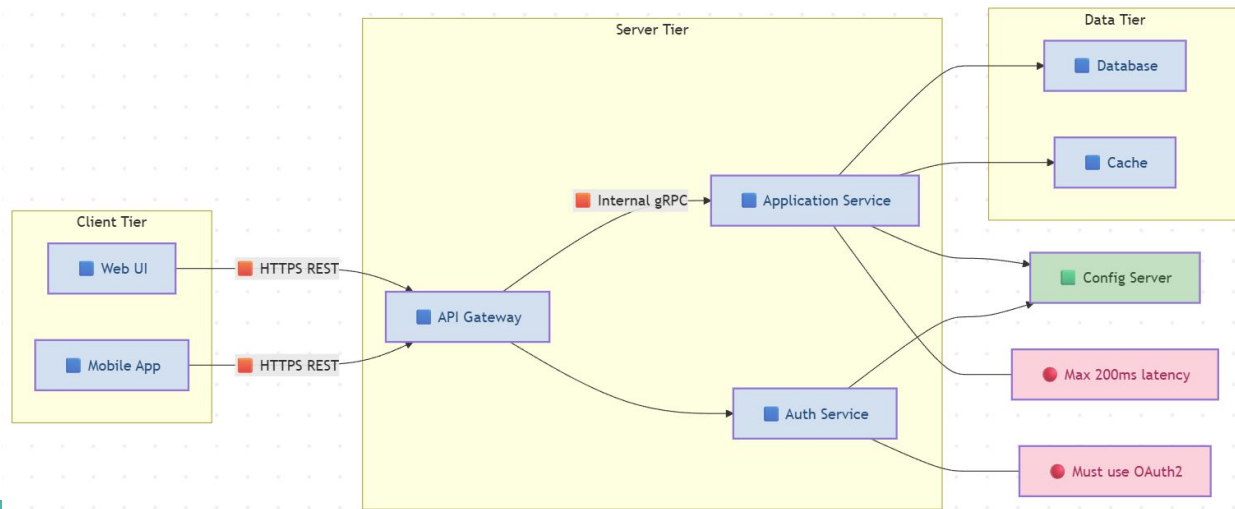
- Not just what components exist — but why they exist, and how they interact
- Key concept: Architectural Decisions
  - What to centralize vs. decentralize
  - What tech to use
  - What to make resilient or scalable





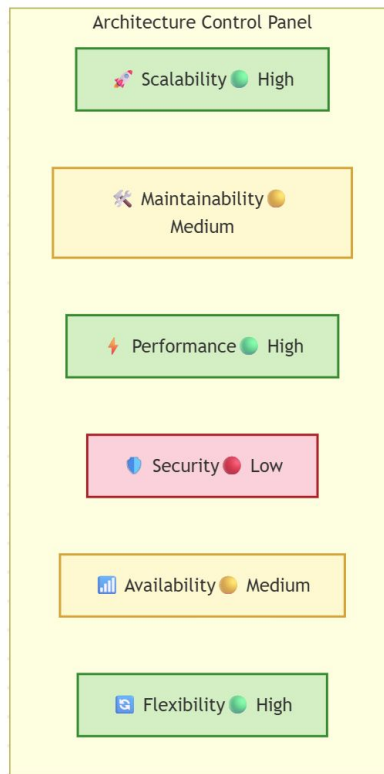
# Key Elements of Software Architecture

- Components (e.g., services, modules)
- Connectors (e.g., APIs, message queues)
- Configuration (deployment, runtime behavior)
- Constraints (performance, cost, latency)



# Architecture's Impact on Quality

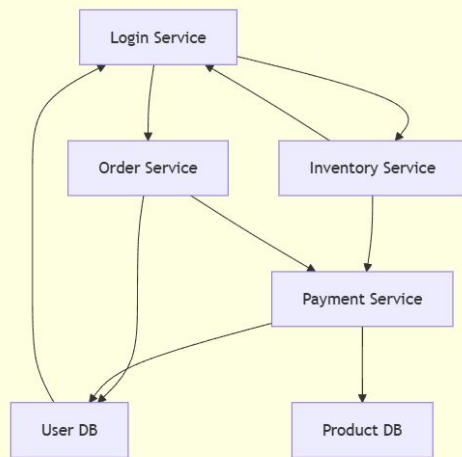
- Architecture influences:
  - Performance: latency, throughput
  - Scalability: horizontal/vertical
  - Availability: failure tolerance
  - Security: surface area, boundaries
  - Maintainability: modularity, testability



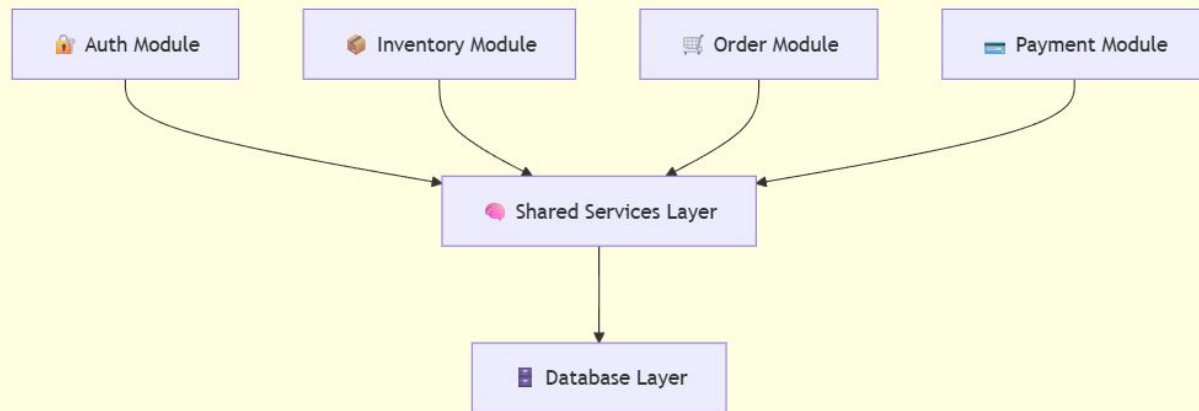
# Poor Architecture – Real-World Consequences

- Tight coupling → hard to scale
- Lack of boundaries → security holes
- Overengineering → complexity & cost
- No documentation → hard onboarding

🤔 Before



😊 After



# Summary & Takeaways

- Architecture is the blueprint of a system
- Influences quality, agility, and success
- Exists across the SDLC — not a one-time effort
- Involves decision-making, trade-offs, and communication
- What's next:
  - Architecture vs. Design vs. Code

---

# Architecture vs. Design vs. Code

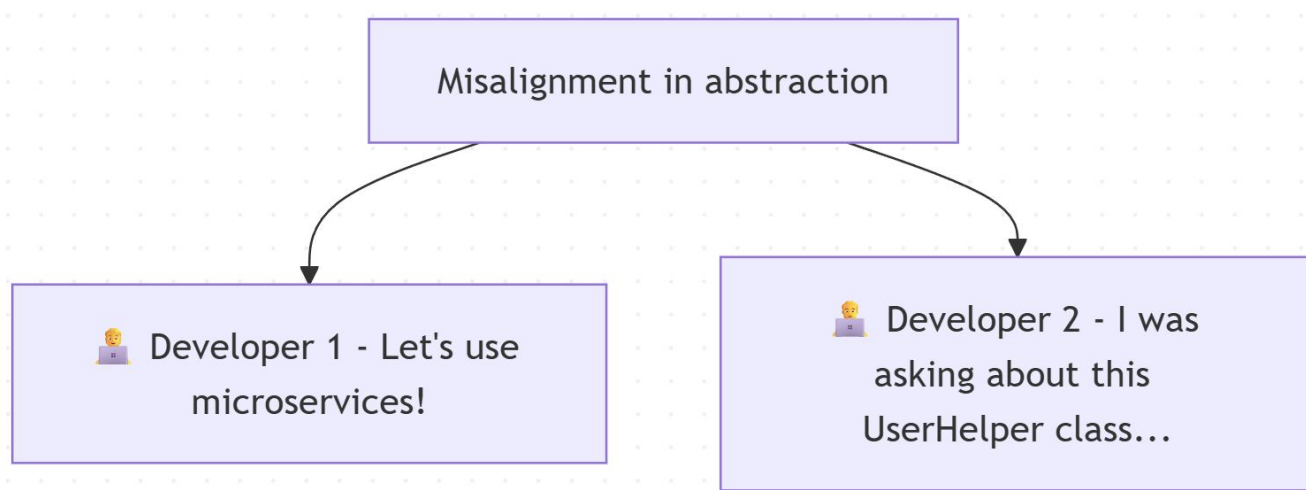
## Foundations of Software Architecture

Mastering Software Architecture Patterns and System Design

---

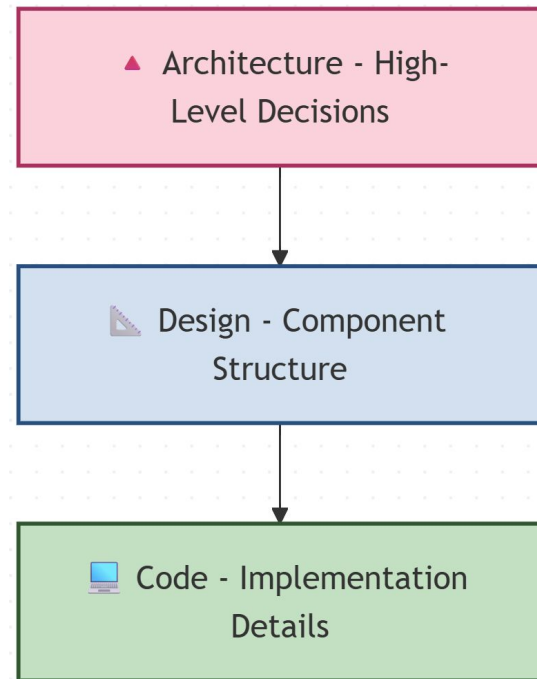
# Why This Matters

- Miscommunication happens when abstraction levels are confused.
- Teams often debate solutions at the wrong level (e.g., microservices vs. helper class).
- Clarity helps avoid over engineering or under engineering.



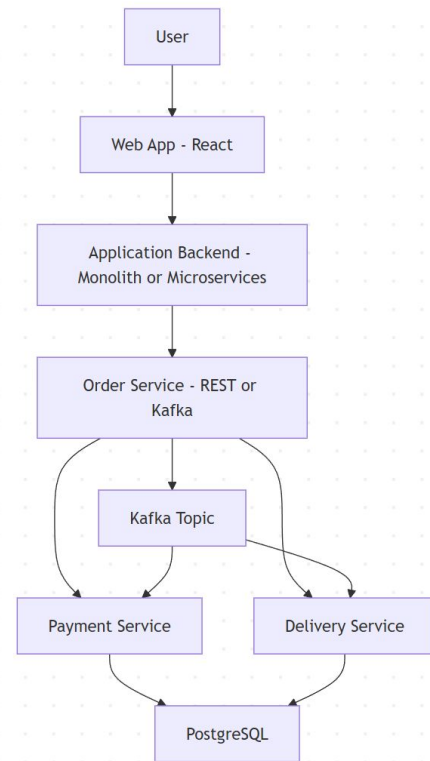
# The Three Levels

-  **Architecture**
  - High-level structure of the system
  - Focuses on components, boundaries, and how they interact
  - Addresses non-functional requirements (scalability, security, etc.)
  - Guides long-term decisions
  - Audience: Architects, Tech Leads
-  **Design**
  - Mid-level organization of modules and logic within components
  - Defines patterns like MVC, layering, dependency injection
  - Specifies data flow, error handling, and responsibilities
  - Bridges architecture and implementation
  - Audience: Developers, Leads
-  **Code**
  - The lowest level — implementation of design
  - Includes classes, functions, interfaces, tests
  - Follows coding standards and implements logic
  - Changes frequently and needs maintainability
  - Audience: Developers



# Architecture – The Big Picture

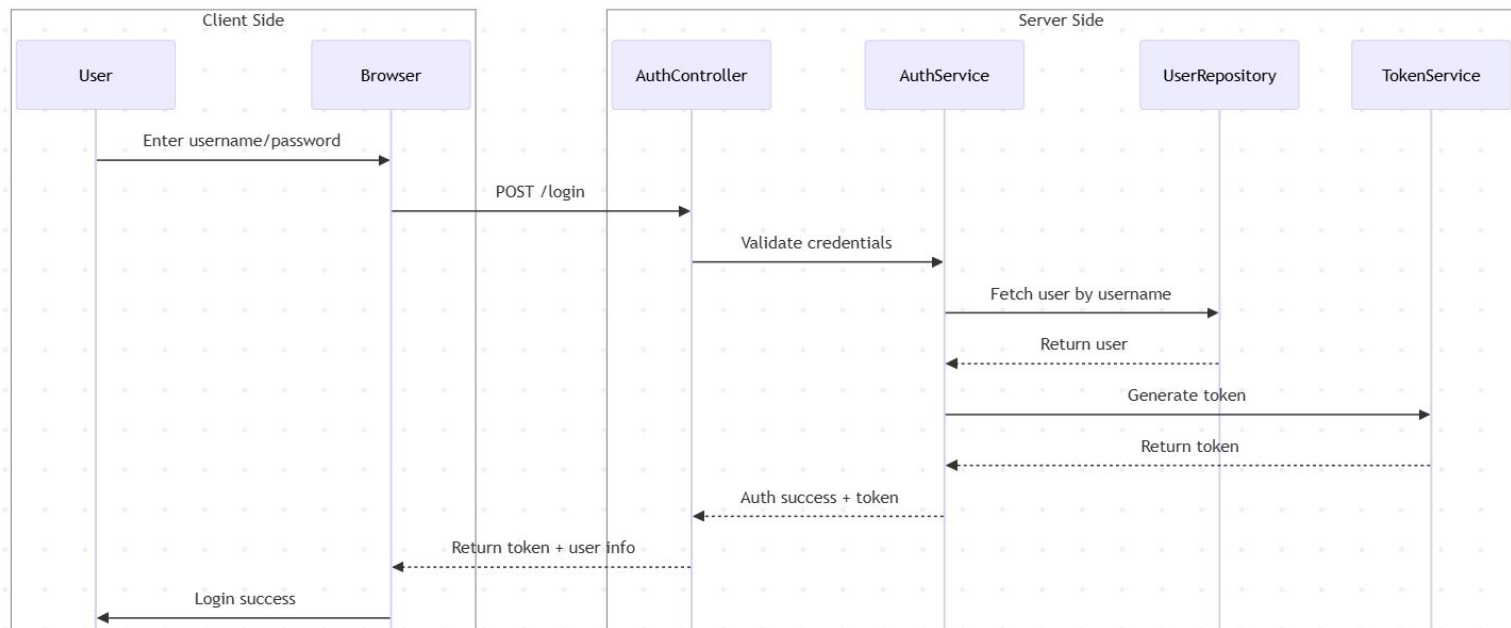
- Defines what parts exist, how they interact, and why they're structured that way.
- Concerned with scalability, deployment, constraints, team boundaries.





# Design – The Blueprint for Devs

- Focused on how individual components are implemented and interact.
- Common topics: patterns (e.g., MVC), error handling, state management.



# Code – The Final Layer

- This is where everything becomes real — source files, functions, classes, tests.
- Reflects decisions made at the design and architectural levels.

```
public class UserService
{
    private readonly IUserRepository _userRepository;
    private readonly IEmailSender _emailSender;

    public UserService(IUserRepository userRepository, IEmailSender emailSender)
    {
        _userRepository = userRepository;
        _emailSender = emailSender;
    }

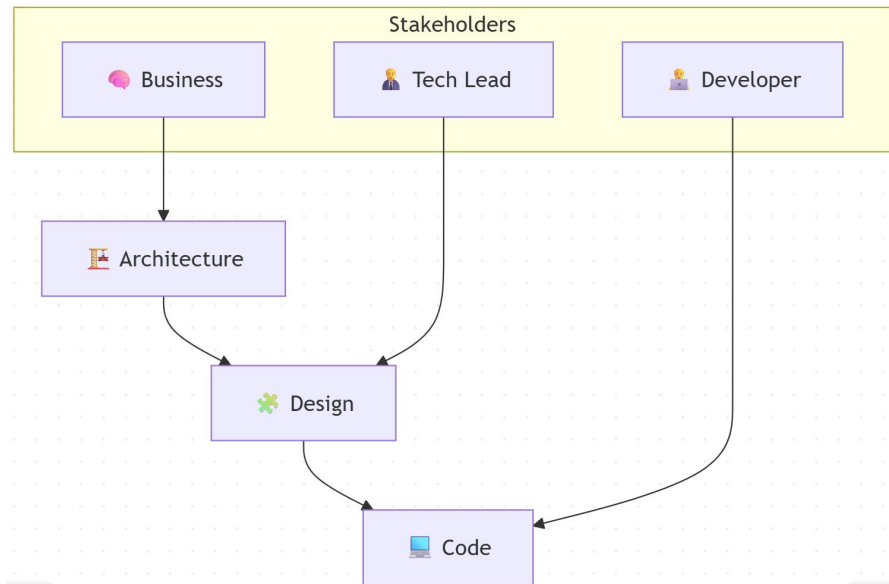
    public async Task RegisterUser(UserDto dto)
    {
        var user = new User(dto.Email, dto.Password);
        await _userRepository.Save(user);
        await _emailSender.SendWelcomeEmail(user.Email);
    }
}
```

# Real-World Example Comparison

-  Architecture
  - Microservices split by domain: Orders, Payments, Restaurants, Delivery
  - Communication via REST APIs or messaging queues
  - Security boundaries between public and internal services
  - Cloud-native deployment with scalability zones
-  Design
  - Payment service uses Strategy Pattern for multiple payment gateways (Razorpay, Stripe, UPI)
  - Delivery ETA estimation follows a layered pattern with a routing engine module
  - User notification uses observer pattern for order updates via SMS, email
-  Code
  - PaymentGateway.cs has concrete implementations like StripeGateway, UPIGateway
  - NotificationService.send() method handles delivery of updates
  - Unit tests for EstimateETA() logic using mock traffic data
- Takeaway:
  - Each level adds more detail and specificity
  - Clear separation helps scale teams and reduce confusion
  - Abstraction ensures flexibility and clarity across system evolution

# Communication Clarity

- Each abstraction level is meant for a different conversation.
- Good architects communicate across levels and tailor clarity to their audience.



# Summary & Takeaways

- Architecture, Design, and Code operate at different abstraction levels.
- They must align — top-down clarity ensures bottom-up consistency.
- Use the right vocabulary for the right conversations.
- What's next:
  - Quality Attributes

---

# Quality Attributes in Software Architecture

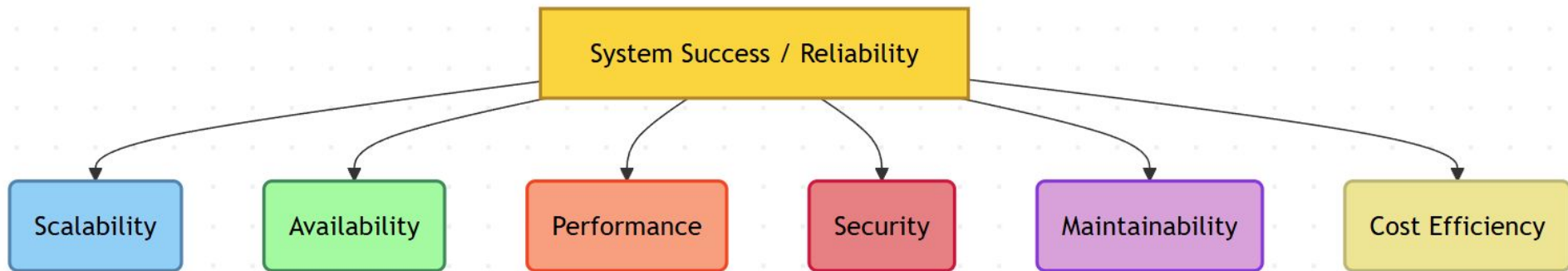
## Foundations of Software Architecture

Mastering Software Architecture Patterns and System Design

---

# What are Quality Attributes?

- Non-functional requirements (NFRs) that define system behavior beyond just features.
- They affect architecture decisions and user experience.
- Often have trade-offs — improving one may weaken another.



# Why Quality Attributes Matter

- Impact long-term success of the system.
- Crucial for choosing patterns, infrastructure, and processes.
- Affects developer productivity, reliability, scalability under load, and security posture.

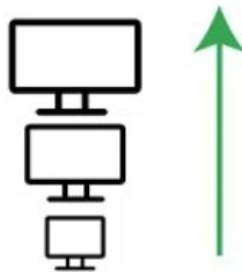




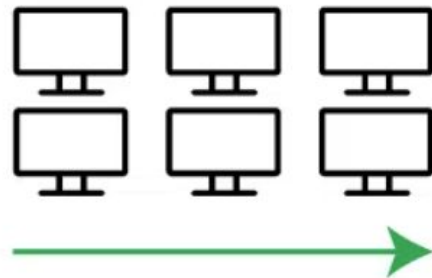
# Scalability – Can the System Grow?

- Ability to handle increased load without performance degradation.
- Types:
  - Vertical Scaling (scale-up)
  - Horizontal Scaling (scale-out)
- Examples:
  - Adding servers behind a load balancer
  - DB sharding for distributed load

**VERTICAL SCALING**  
Increase size of instance  
(RAM, CPU etc.)



**HORIZONTAL SCALING**  
(Add more instances)



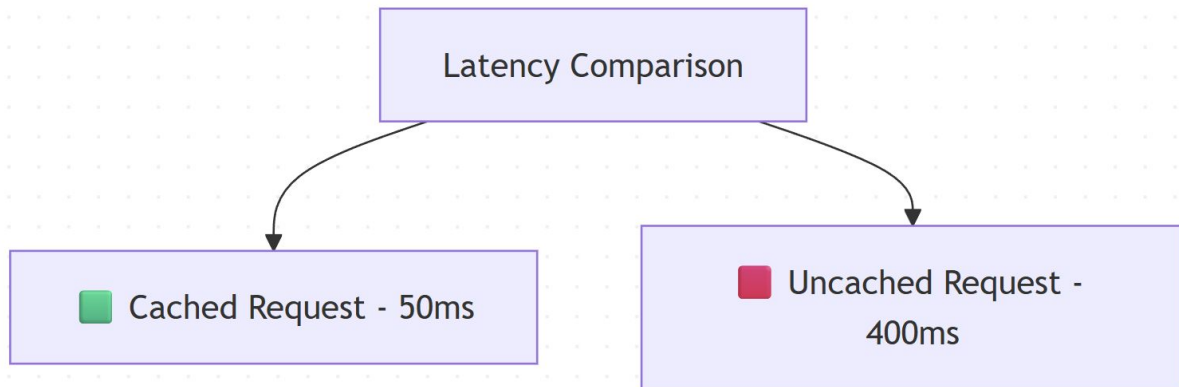
# Types of Scalability

- Read Scalability – CDN, caching (Redis, Memcached)
- Write Scalability – DB partitioning/sharding, async queues
- Elastic Scalability – Auto-scaling on cloud (AWS/GCP)



# Performance – How Fast is the System?

- Speed at which system processes requests.
- Includes:
  - Latency (response delay)
  - Throughput (requests/sec)
  - Load time, batch processing time, etc.
- Optimized via caching, compression, indexing, CDN, etc.



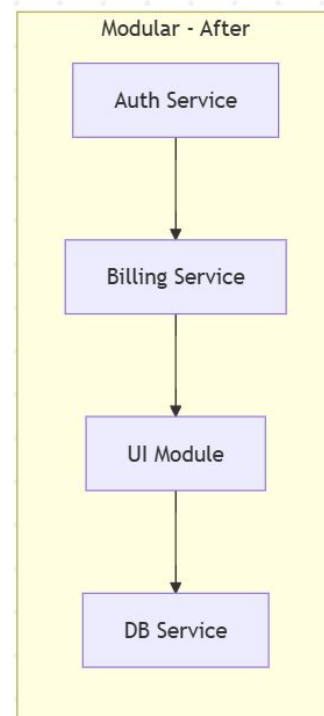
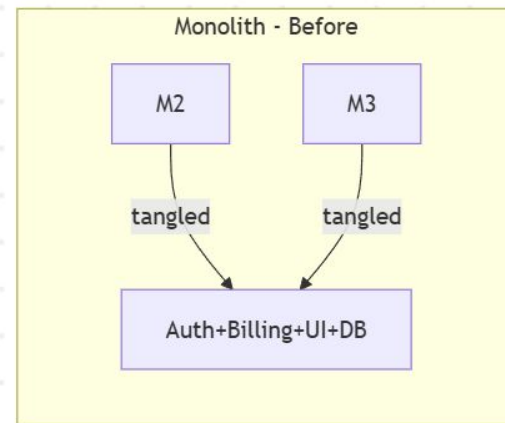
# Availability – Is the System Up?

- Uptime of the system (measured in 9s: 99.9%, 99.99%)
- Downtime due to:
  - Server crashes
  - DB failures
  - Network outages
- Patterns: Redundancy, failover, heartbeat checks, retries



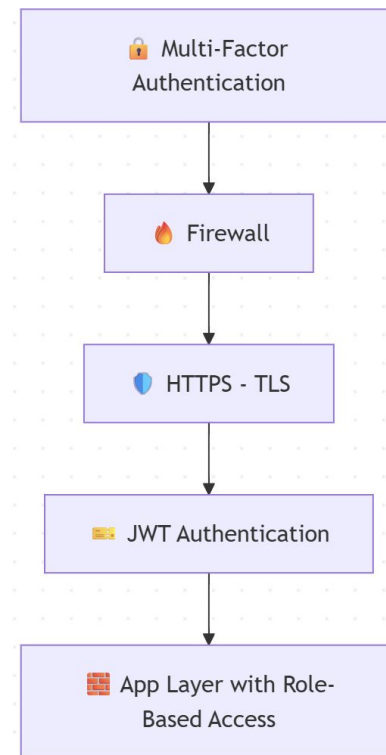
# Maintainability – Can We Evolve the System?

- Ease of modifying the system with minimal risk.
- Factors:
  - Code structure (modularity, readability)
  - Automated testing, CI/CD
  - Documentation
- Monoliths vs. Microservices trade-off



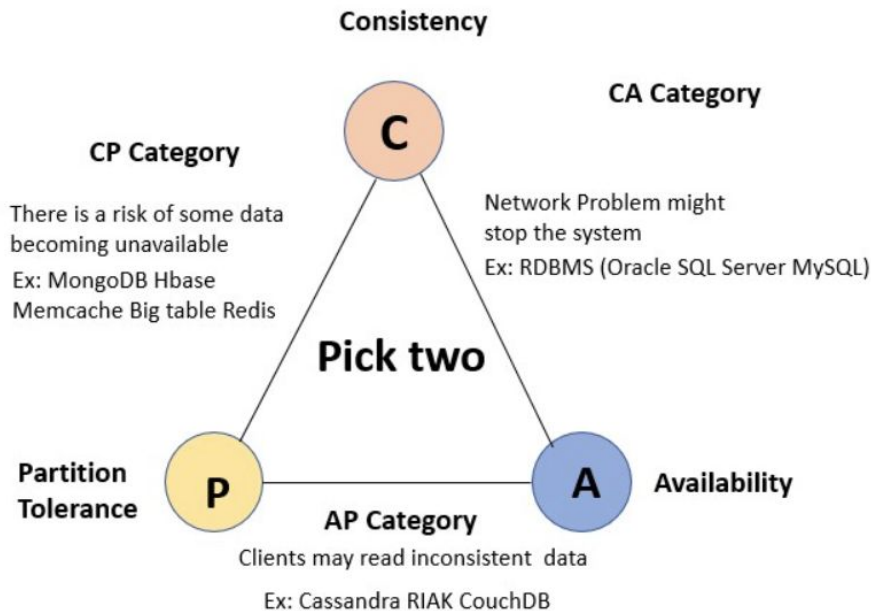
# Security – Is the System Safe?

- Protecting system from unauthorized access & breaches.
- Consider:
  - Authentication & Authorization
  - Data encryption (at rest/in transit)
  - Secure coding practices
  - Input validation, rate limiting
- Zero trust model & least privilege principle



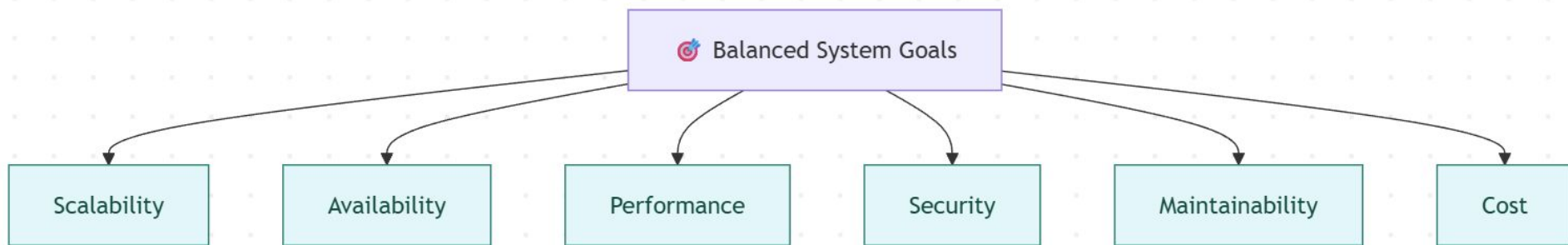
# Trade-offs Between Quality Attributes

- CAP Theorem: Consistency vs. Availability vs. Partition Tolerance
- Other trade-offs:
  - Performance vs. Security (e.g., encryption overhead)
  - Maintainability vs. Performance (e.g., abstraction layers)



# Balancing Quality Attributes

- No “one-size-fits-all”
- Driven by:
  - Use case
  - Business goals
  - User expectations
- Prioritize: e.g., for healthcare → security > performance





# Summary

- Quality attributes shape your system's resilience, growth, and longevity.
- Each attribute needs active design consideration.
- Trade-offs are inevitable — understand your priorities.
- What's next:
  - Role & Responsibilities of a Software Architect

---

# Role & Responsibilities of a Software Architect

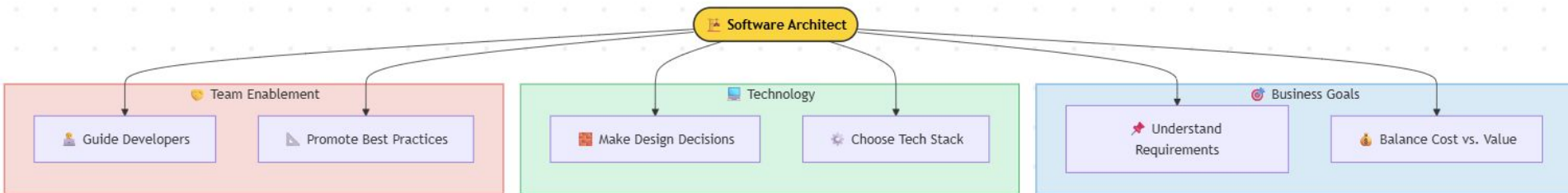
## Foundations of Software Architecture

Mastering Software Architecture Patterns and System Design

---

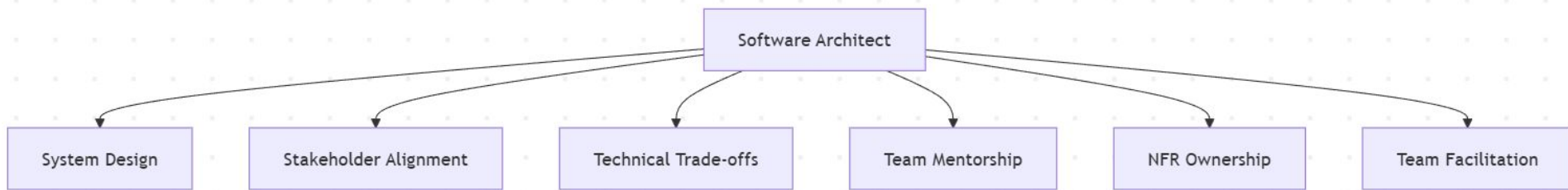
# Who is a Software Architect?

- A software architect defines the technical direction of a system.
- Bridges the gap between business goals and engineering execution.
- Makes design decisions with long-term impact.



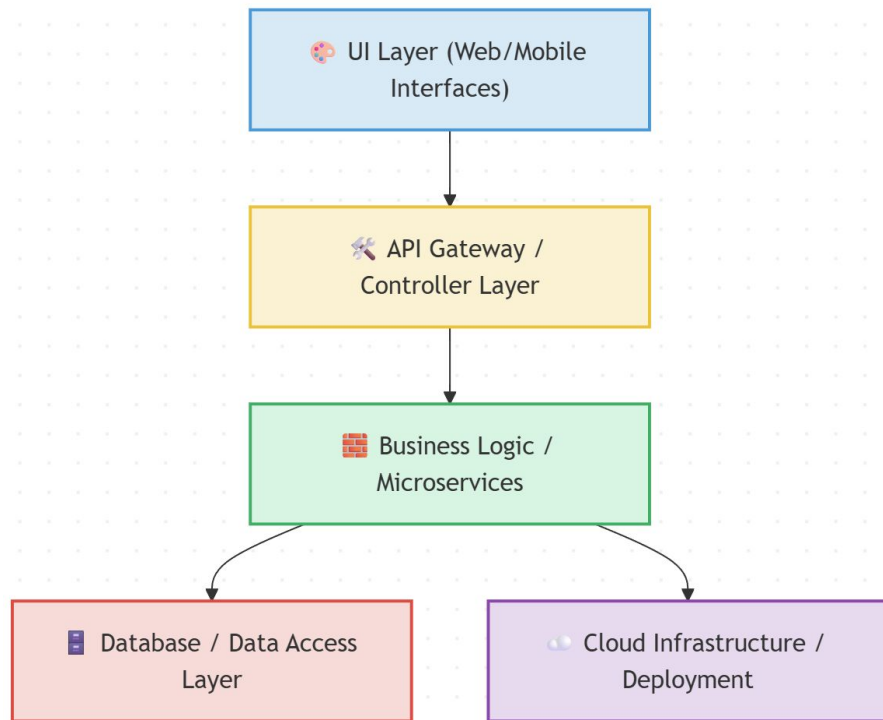
# Core Responsibilities at a Glance

- System design & architecture decisions
- Making trade-offs
- Aligning with stakeholders
- Facilitating the team
- Ensuring non-functional requirements (NFRs)
- Mentoring developers



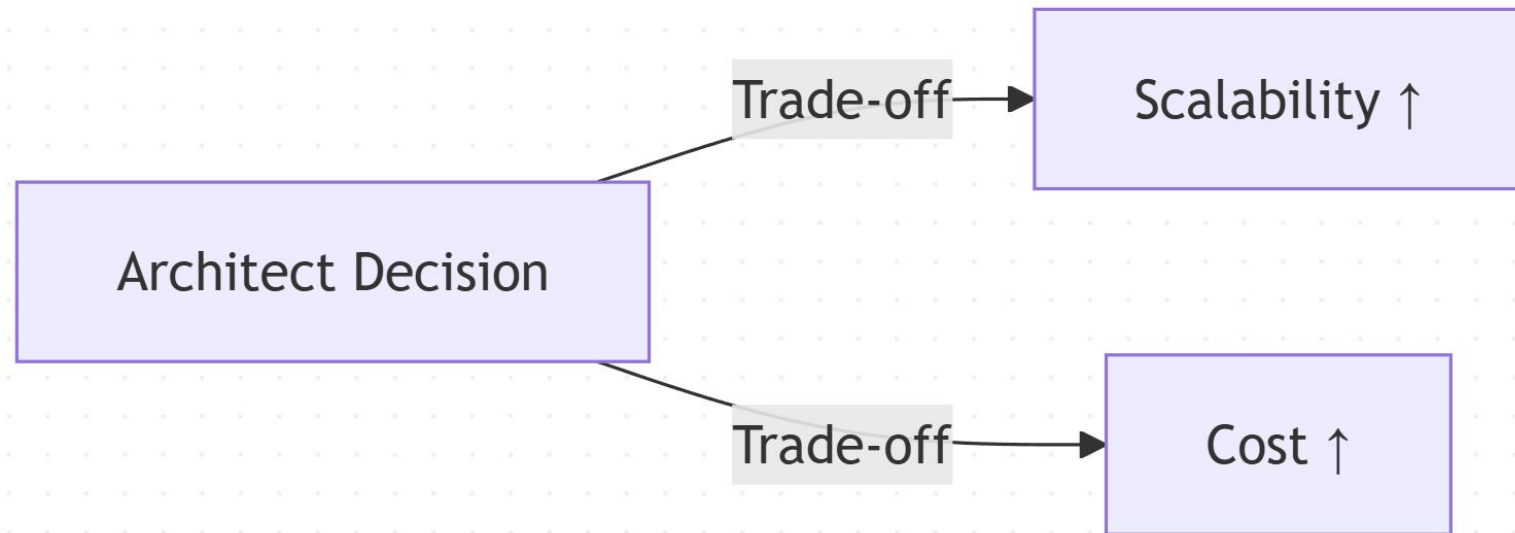
# Responsibility – System Design

- Define system boundaries and components
- Choose the right architecture pattern
- Make foundational decisions (APIs, data flow, deployment model)
- Ensure scalability, performance, and maintainability



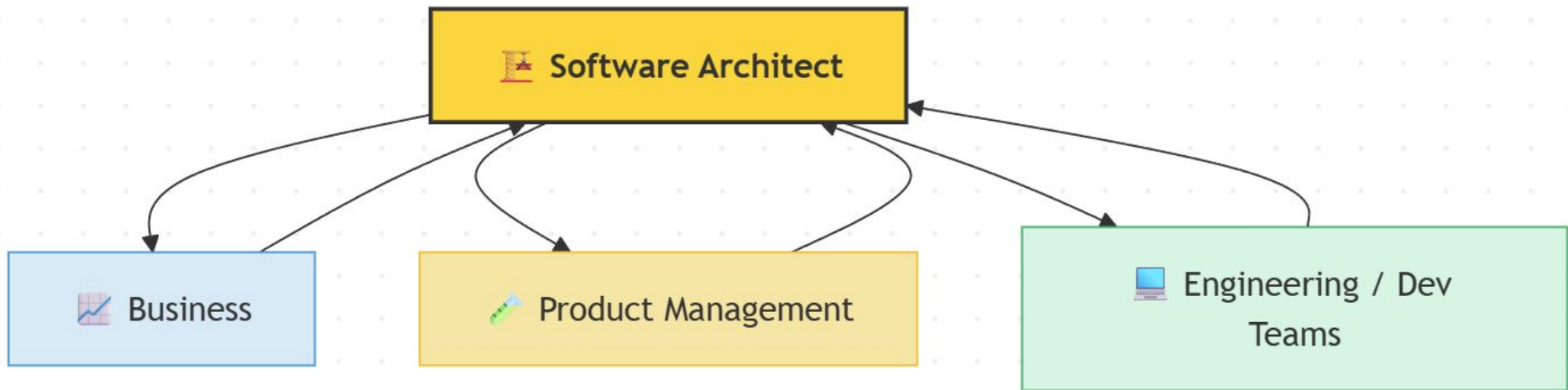
# Responsibility – Technical Trade-offs

- No perfect solution — every decision has trade-offs
- E.g., Performance vs Maintainability, Speed vs Accuracy
- Architect's role is to analyze impact and decide wisely



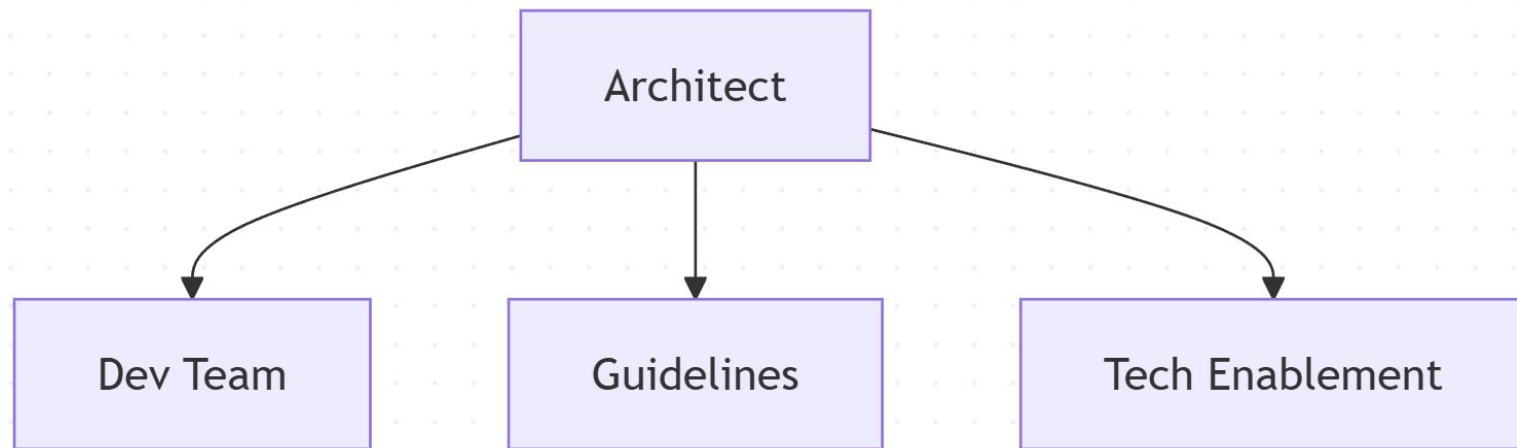
# Responsibility – Stakeholder Alignment

- Work with:
  - Product managers (features, priorities)
  - Business leaders (goals, cost)
  - Developers (feasibility)
- Translate requirements into architecture decisions



# Responsibility – Enabling the Team

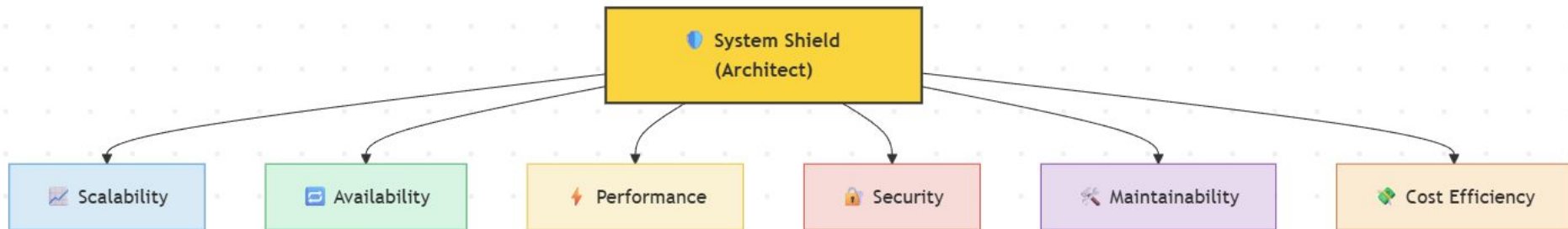
- Define clear architecture boundaries
- Provide reusable patterns and guidelines
- Reduce decision fatigue for developers
- Unblock and support engineering team





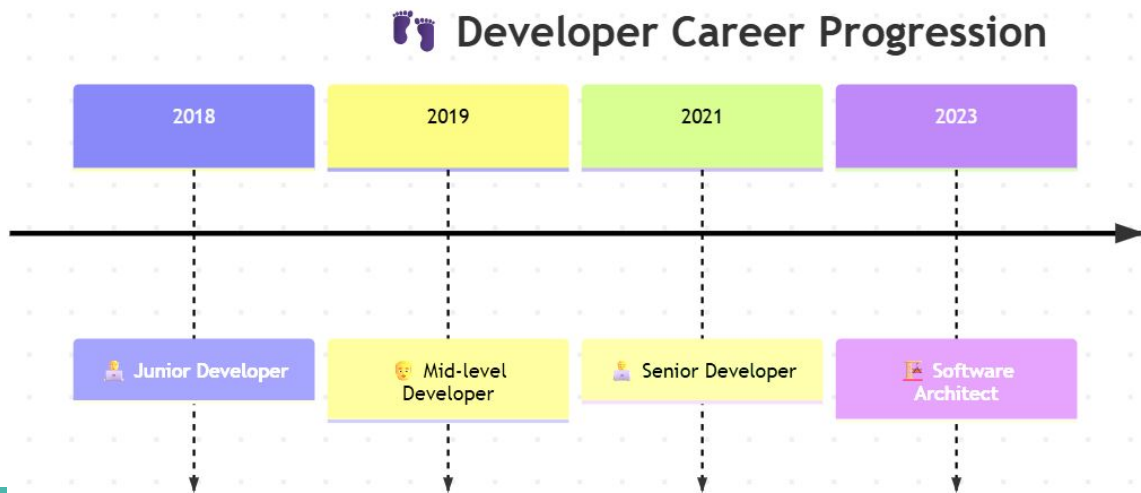
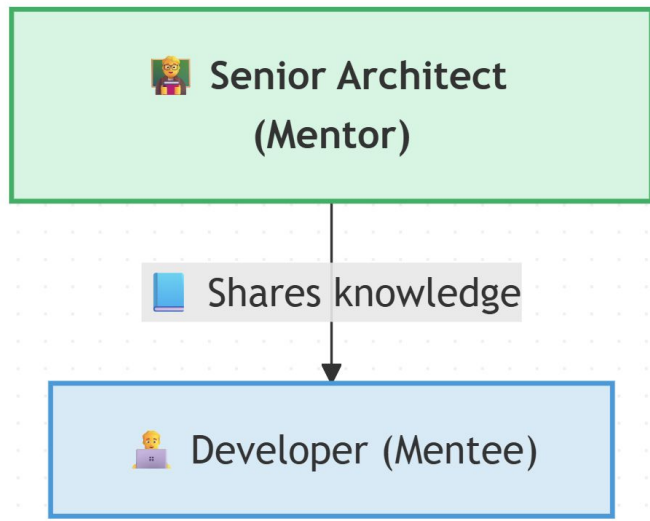
# Responsibility – Owning NFRs

- Architect ensures that scalability, performance, availability, security, etc. are part of the design
- NFRs often ignored in early stages — architect must champion them



# Responsibility – Mentorship

- Help developers grow technically
- Review designs & PRs
- Share architectural reasoning
- Foster a culture of clean code & scalable thinking

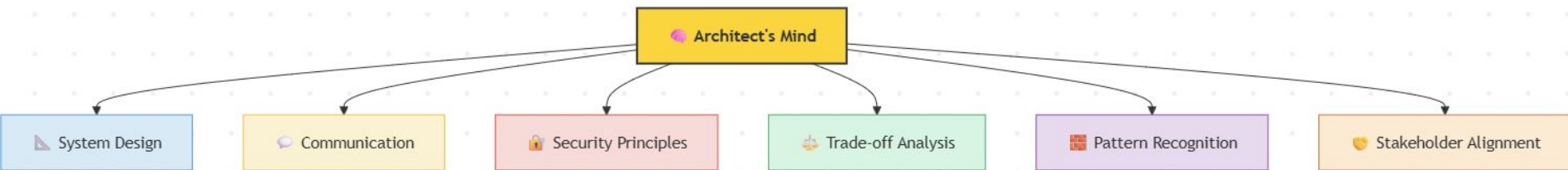


# Architect vs Tech Lead

- Focus:
  - **Architect:** System-wide design and long-term technical vision
  - **Tech Lead:** Day-to-day execution, sprint planning, code delivery
- Scope:
  - **Architect:** Works across teams, components, and sometimes products
  - **Tech Lead:** Focuses on a specific team or feature group
- Primary Outputs:
  - **Architect:** Diagrams, architectural blueprints, tech strategies
  - **Tech Lead:** Clean, maintainable code, mentoring, backlog execution
- Stakeholder Interaction:
  - **Architect:** Collaborates with product, business, ops, and engineering
  - **Tech Lead:** Works closely with devs and PMs
- Decision Style:
  - **Architect:** Strategic, long-term decisions and trade-offs
  - **Tech Lead:** Tactical, short-term technical leadership
- Overlap:
  - Both may mentor developers, conduct code reviews, and influence design
  - In smaller teams, roles may blend — one person may do both

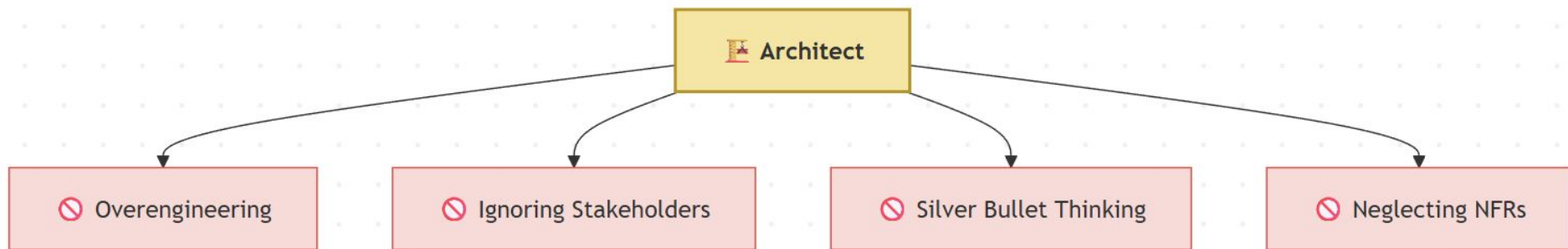
# Key Skills of a Great Architect

- Deep technical knowledge
- System-level thinking
- Strong communication
- Decision-making under ambiguity
- Business and domain awareness
- Empathy for users and developers



# Common Pitfalls to Avoid

- Over-engineering
- Not involving the team
- Ignoring feedback loops
- Losing touch with the code
- Not evolving the architecture



# Summary

- Architects define how a system should be built
- Responsibilities go beyond just diagrams — they involve people, process, and alignment
- It's about leading through design, not control
- A great architect enables the system and the team to succeed
- What's next:
  - Documenting Architecture: ADRs, C4 Diagrams & Best Practices

---

---

# Documenting Architecture

## ADRs, C4 Diagrams & Best Practices

— Foundations of Software Architecture —

Mastering Software Architecture Patterns and System Design

---

---

# Why Document Architecture?

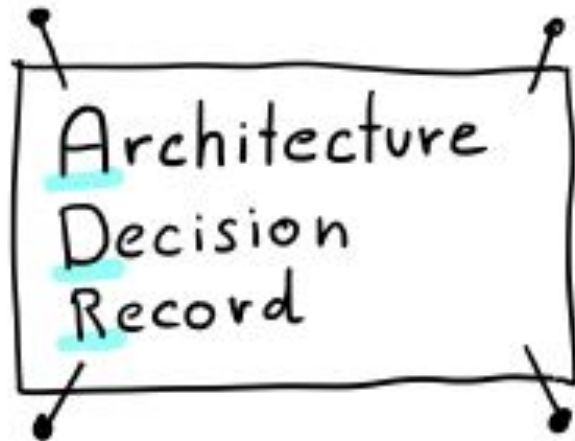
- Keeps decisions visible and traceable
- Helps onboard new developers
- Improves collaboration and review
- Essential for audits, scaling, and handovers





# What Are Architecture Decision Records (ADRs)?

- Lightweight docs capturing important decisions
- Format: Title, Context, Decision, Status, Consequences
- Example decisions: use of PostgreSQL over MongoDB, JWT over Session ID



# ADR Example – Template Breakdown

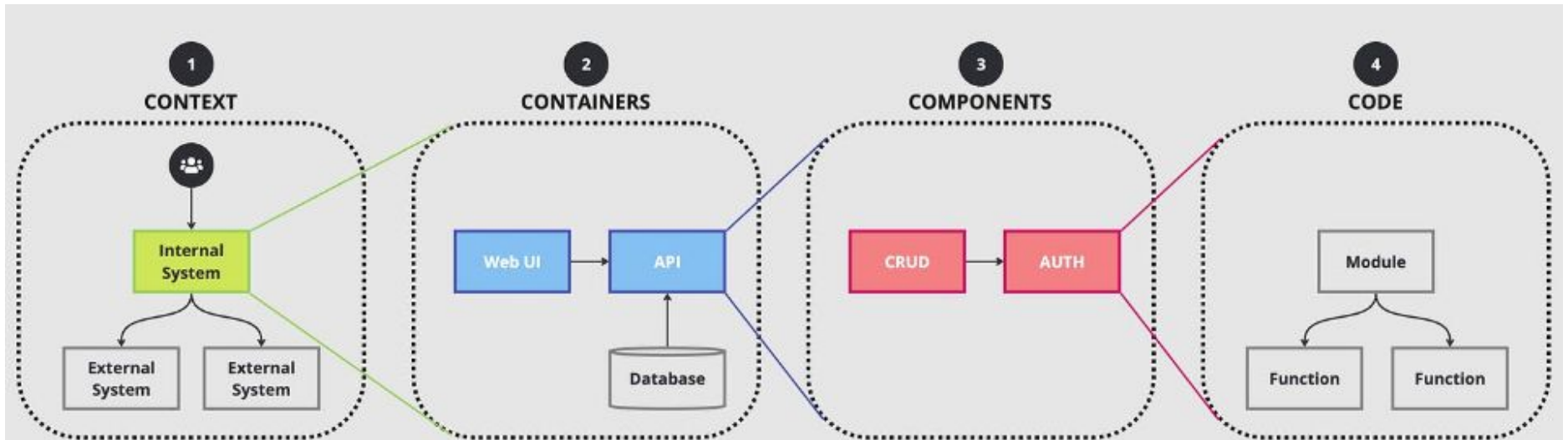
- **Title:** Descriptive title
- **Status:** Proposed, Accepted, Deprecated
- **Context:** What led to this decision
- **Decision:** What was chosen and why
- **Consequences:** What changes, what's impacted

**Architecture Decision Record (ADR) Template**

|                     |                                                                                                  |
|---------------------|--------------------------------------------------------------------------------------------------|
| <b>Title</b>        | Short name of the decision to be made                                                            |
| <b>Date</b>         | Date when the decision was made                                                                  |
| <b>Context</b>      | Explanation of the problem                                                                       |
| <b>Decision</b>     | Clear statement of the decision made                                                             |
| <b>Alternatives</b> | Description of alternative solutions that were considered                                        |
| <b>Consequences</b> | Implications of the decision, positive or negative                                               |
| <b>Participants</b> | Names of team members how made the decision                                                      |
| <b>Status</b>       | The current status of the decision, such as "proposed," "accepted," "rejected," or "superseded." |

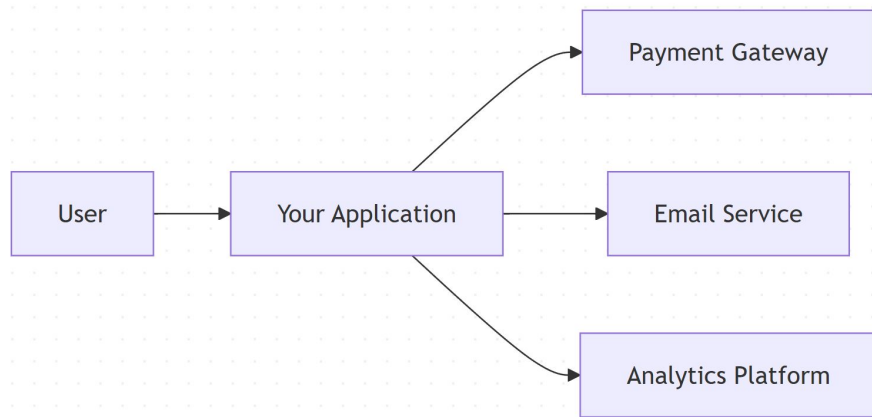
# The C4 Model – Overview

- C4 = Context, Container, Component, Code
- Helps visualize software at different levels of abstraction
- Focuses on static structure



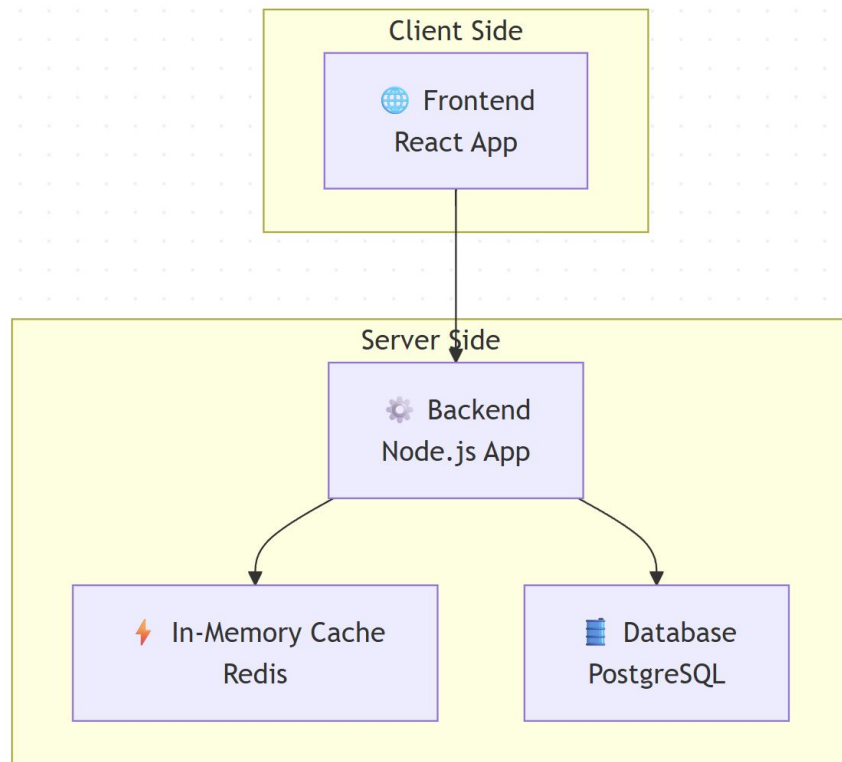
# C4 Level 1 – Context Diagram

- Big picture: How your system fits into the environment
- Shows users, external systems, and primary interfaces



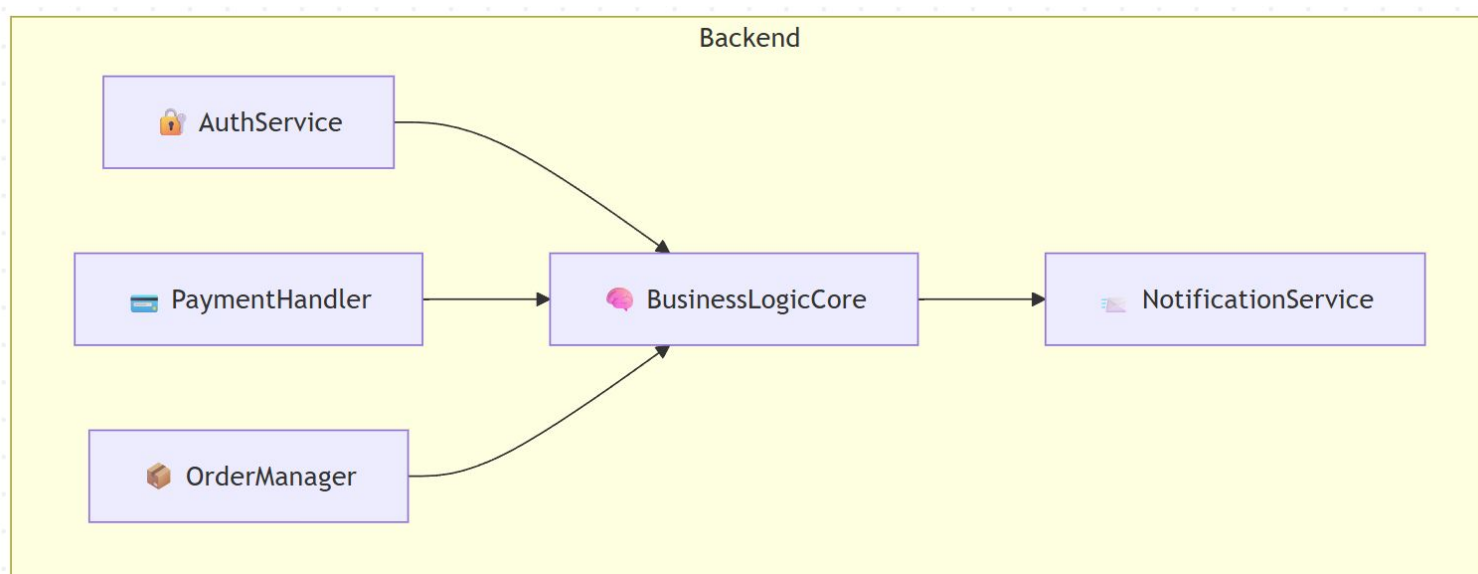
# C4 Level 2 – Container Diagram

- Zoom into your system
- Shows major building blocks: Web App, API, DB, Services
- Describes tech used (e.g., React frontend, Node.js backend)



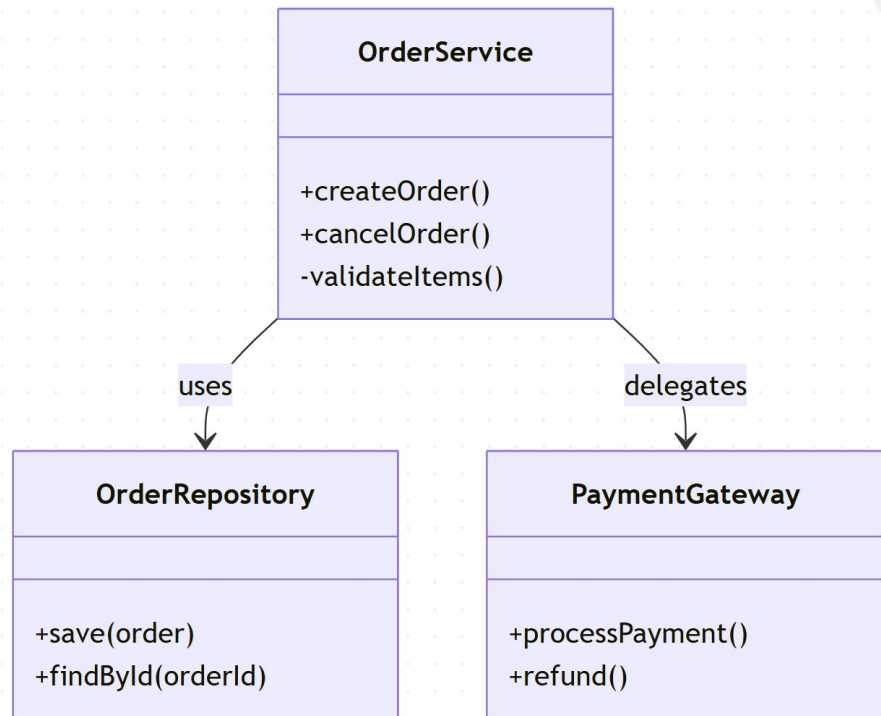
# C4 Level 3 – Component Diagram

- Inside a container: what components/modules exist?
- Shows how services, controllers, and libraries interact



## C4 Level 4 – Code (Optional)

- Lowest level: class diagram or package diagram
- Often too detailed for regular docs
- Use sparingly, mainly for teaching/onboarding



# Tools for Diagramming Architecture

- Structurizr (C4 as code)
- PlantUML / Mermaid (lightweight, markdown-friendly)
- Draw.io, Lucidchart (drag-drop GUI)
- Archi (for enterprise modeling, ArchiMate)



# Best Practices for Architecture Docs

- Keep them versioned with code (in Git)
- Don't aim for perfection — evolve over time
- Keep diagrams simple & focused
- Use ADRs for reasoning + diagrams for structure
- Keep context fresh with timestamps and status

# Summary

- Good documentation tells the why and the what
- ADRs capture key decisions transparently
- C4 diagrams provide layered clarity of the system
- Use the right tools and keep it lightweight
- What's next:
  - Layered & Modular Architecture Patterns